Banking Transaction Categorisation

One transaction's journey through a plain-PHP backend

Dean Norman · SOUS senior engineer assessment · October 2026

The brief, and where each line is answered

Requirement	Where
Accept and categorise transaction history (CSV)	`POST /transactions`; CSV as source material for rules and fixtures
Ten predefined categories	`Category` enum, in the brief's order
API to submit, list, fetch by id	the three endpoints, `docs/api.md`
AI-powered categorisation, prompts in the repo	rules derived by Claude Fable 5.1, `docs/prompts/`
Language and framework, with reasoning	plain PHP 8.3 + MariaDB, `docs/language-choice.md`
Docker Compose	three services, port 8090
Documentation with trade-offs	`docs/trade-offs.md`, three ADRs
Unit and integration tests	93 tests inside the stack, plus a smoke script
Small, logical commits	eight steps, eight commits, eight documents

What the data said before I wrote any code

1000 rows, 20 distinct descriptions

Albert Heijn, AH Online, NS, Car Lease, Eneco, Ziggo, T-Mobile, Netflix, Bol.com, Rent, Municipal Tax, plus salary, tax refund, transfers, ATM, PayPal, iDEAL, credit-card settlement.

Three Dutch banks, no duplicate ids, no malformed rows.

One catch

The sign of `amount` disagrees with `type` in about half the rows: negative credits, positive debits.

Decision: store both exactly as given, never let direction influence the category.

ADR 0003

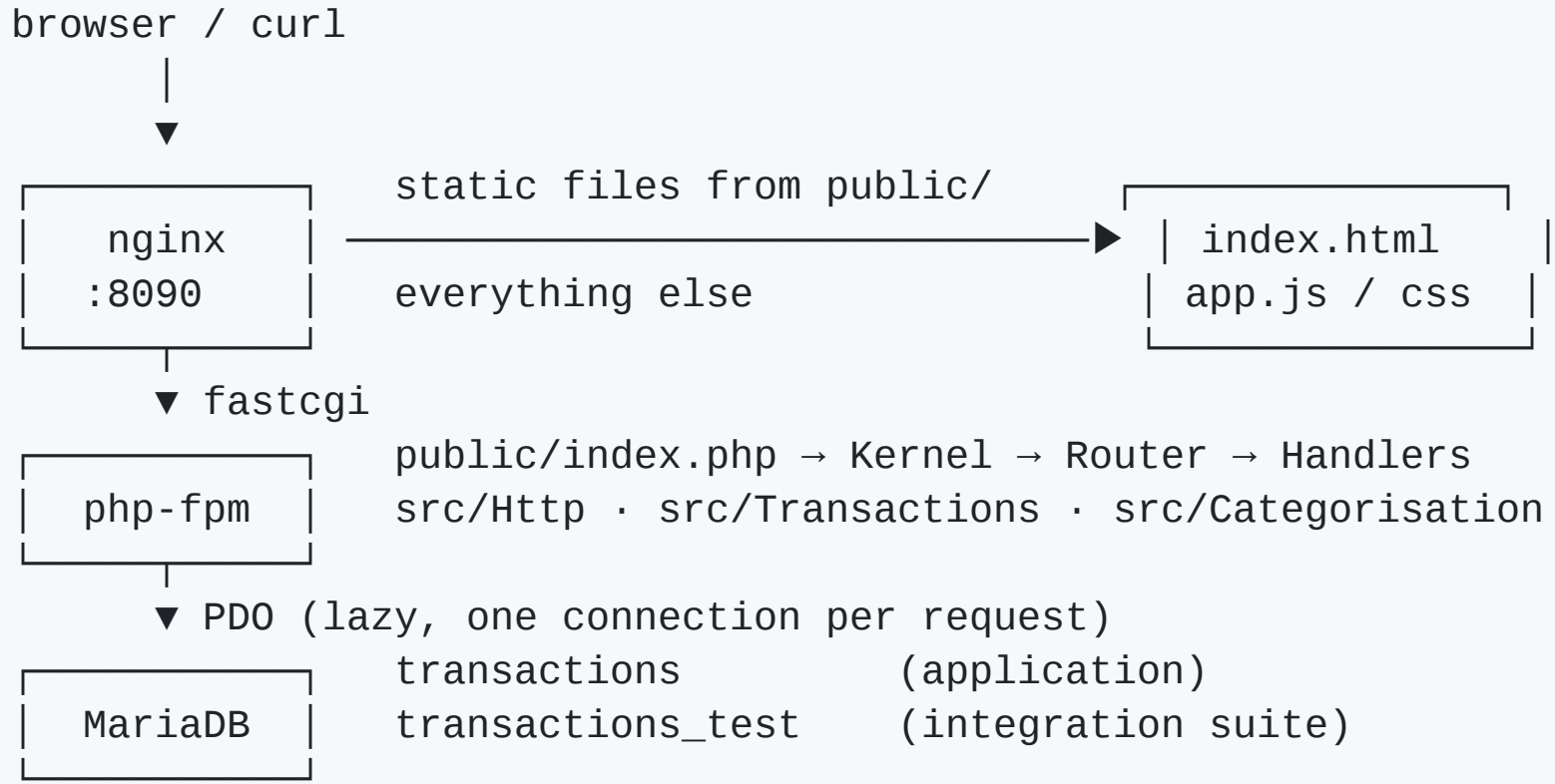
Twelve decisions before the first commit

I had the AI tool interview me, one decision at a time, with a recommendation for each. The record is `docs/prompts/00-design-interview.md`.

- rules, not a runtime LLM
- plain PHP, one front controller
- CSV is source material
- server-minted ids + unique reference
- amounts stored as given
- keyword rules, first match, Miscellaneous fallback
- one row, one insert
- minimal list, cap 500
- static page, same origin
- integration tests on a second database
- one commit per step
- eight steps

Every later choice traces back to one of these.

The stack



Three containers, one `docker compose up -d --build`, nothing on the host but Docker.

The journey of one transaction

```
POST /transactions
   │
   ├─ Request::fromGlobals        decode JSON, remember if it was malformed     400
   ├─ Router                      method + path, {id} segments                  404 / 405
   ├─ TransactionValidator        every field checked, all failures at once     422
   ├─ RuleBasedCategoriser        first matching rule wins, else fallback
   ├─ TransactionRepository       mint UUID, one INSERT, unique reference       409
   └─ Response::ok(201)           the stored row, same shape as every GET
```

Each line is a step, a commit and a document in `docs/journey/`.

Step 2 · Validation says everything at once

```
POST {"amount": "lots", "type": "wire"}

422 {"error": {"code": "validation_failed",
               "message": "One or more fields are invalid",
               "fields": {"amount": "must be a decimal number such as -87.18",
                          "timestamp": "is required",
                          "description": "is required",
                          "type": "must be debit or credit",
                          "account_number": "is required"}}}
```

- Field names are the CSV columns in snake_case; unknown fields are ignored
- Money is a decimal string from the moment it is normalised, never a float downstream
- The bank's `2024-05-16 07:22:18.808433` and ISO 8601 both accepted; no zone means UTC

Step 3 · How a category is decided

21 ordered rules, 9 categories, one fallback

- Case-insensitive, word-bounded patterns
- First match wins; Dining Out sits before Transportation so `Uber Eats` beats `Uber`
- No rule maps to Miscellaneous: it is only ever the fallback, and the source says so
- The description is the only input

On the supplied 1000 rows

Category	Rows
Miscellaneous	444
Utilities	146
Transportation	114
Housing	101
Groceries	95
Entertainment	51
Shopping	49

Dining Out, Healthcare, Education: rules exist, no examples in the data.

How the AI coding tool was used

Prompt 0: the design interview. The brief and the CSV, with the instruction to look up facts in the files, ask one decision at a time, and recommend an answer. Twelve questions, twelve answers, every one implemented.

Prompt 1: deriving the rule table. The data profile (20 descriptions with counts) and the ten categories, with constraints: word boundaries, no Miscellaneous rule, cover the three empty categories, stable ids, call out ambiguities and ordering.

The answer is transcribed one to one into `rules.php`. The tests pin all 20 descriptions, the whole fixture and the distribution above, so the transcription cannot drift.

What was not done: no LLM at runtime. `Categoriser` is an interface; the LLM implementation is the empty slot behind it. ADR 0001 says why.

Step 4 · Storage, and the two things the database enforces

```
INSERT INTO transactions (id, reference, amount, occurred_at, description, type,
  account_number, category, categorisation_source, categorised_at, created_at)
```

- The server mints the id (UUID v4). The bank's `TXN00001` is kept as `reference`, optional and unique. A repeat is `409`; a submission without a reference is always accepted. ADR 0002
- One row, one statement. Transaction and categorisation together; no pending state.
- **`DECIMAL(14,2)`** keeps money exact; **`DATETIME(6)`** round-trips the bank's microseconds; the unique key is what turns a duplicate into a `409`, not application code.
- The connection is lazy. Wiring never touches the database; an outage is a `503` from `/health` and a JSON `500` from a write, never a blank page.

Step 5 · Reading it back

Request	Answer
`GET /transactions/{id}`	the same JSON the POST returned, byte for byte
`GET /transactions/TXN00001`	`404`, the id shape is not a contract clients are told
`GET /transactions`	newest first, at most 500, `meta: {count, cap, category}`
`GET /transactions?category=Housing`	one indexed `WHERE`; the index was created in step 1 for this
`GET /transactions?category=Pets`	`422`, with the ten valid names

No paging, on purpose. The cap says so in `meta.cap`; paging is the first item on the "next" list.

Step 6 · The journey in a browser

One static page from `public/`, same origin as the API, no build step.

- Left: the POST form, with sample rows from the history plus two the history does not contain
- Middle: the list, the category filter, the count
- Right: fetch by id; filled automatically after a submit or a row click

Every response shown is the API's own JSON. The page adds nothing the API does not say, including the `422` field map and the `409`.

Step 7 · Proof

Suite	What it drives	Count
Unit	router, request, response, validator, rule table, categoriser, UUID, serialisation	72 tests
Integration	the real `Kernel` against `transactions_test` on the same MariaDB	21 tests
Smoke	14 requests through nginx and PHP-FPM, status codes	`scripts/smoke.sh`

```
docker compose run --rm php vendor/bin/phpunit     # 93 tests, 334 assertions
scripts/smoke.sh                                    # smoke passed
```

The integration base class refuses to run if the test database is the application database. Every test starts from an empty table. The `Clock` is held still so timestamps are asserted exactly.

Trade-offs I would defend

Chose	Gave up	Next step
Rules at runtime	unseen descriptions	LLM behind `Categoriser` for the fallback case
Category written once, on submit	rule changes reach old rows	`categorisations` history table + re-run job
Reference-based idempotency	protection when the client omits it	nothing; fingerprinting merges real duplicates
Cap of 500, no paging	large histories	cursor on `(created_at, id)`
Init SQL, no migration tool	schema changes after first start	migrations applied on deploy
PHPUnit in the image	image size	a `test` build target
No auth, no rate limit	production readiness	kernel middleware

Why plain PHP and MariaDB

- It is the stack I ship daily; the brief asked for the one I am most comfortable with
- No framework between the reader and the decisions: the router is forty lines, the validator says what each field must be, the kernel wires four handlers by hand
- PHP 8.3 as it is now: enums, readonly, named arguments, first-class callables, strict types everywhere
- One deliberate upgrade on the shape I work in: a single front controller, so handlers take a `Request` and return a `Response` and can be driven from a test without a web server
- MariaDB because the invariants (unique reference, exact money, microsecond timestamps) belong to the database

Questions I expect, answers I have

- "Walk me through a POST." Slide 6, four calls, in order.
- "What happens if the same transaction is sent twice?" With a reference: 409, nothing stored. Without: two rows, the client's choice.
- "How do you add a category or a merchant?" One line in `rules.php`; the fixture test fails until the expectation table agrees.
- "Why is **`Salary`** Miscellaneous?" The fixed list has no income category; the fallback says so in `categorisation_source`.
- "Where would an LLM go?" Behind `Categoriser`, consulted only on fallback, answers cached as rule candidates for review.
- "What is not production-ready?" Auth, rate limiting, paging, migrations, re-categorisation. All listed, all small, all deliberately absent.

Thank you

github.com/DeanNorman/transaction-categoriser-php

Start at README.md, then docs/journey/01-scaffold.md